

CS 122A Custom Laboratory Project Report Battleship

Hooman Masroor, Joash Gem Marcos

6/10/2026

Introduction

We implemented an embedded gaming system that plays the classic Battleship board game across two devices. It uses wifi to communicate between the two pico setups which send to the screen via SPI to display gameplay. The users first place their ships on a grid and then take turns attacking each other's ships without knowing their location.

We originally wanted to add sprites and animation to the screen using LVGL, but it would have been too time consuming and we did not have enough time to implement it. We tested our code and could not find any obvious bugs, but there is still the possibility of some edge case errors.

Elements of Complexity

Explain the parts of your project that you consider to be elements of complexity and what you think makes your project stand out. Clearly distinguish between elements of complexity that were successfully implemented, those that were not fully implemented, or those that were not implemented at all.

- **Game Logic:**

We wrote the game logic from scratch without the use of any AI. We implemented it as a turn based first and after getting wifi , we converted it to work across two devices.

- **Wifi:**

Our two setups each have a raspberry pi pico which handles wifi communication. One setup acts as a server and the other acts as a client. We thought this was unique because some other teams used wifi, we did not see anyone else using wifi to communicate between 2 picos.

- **Custom LVGL:**

LVGL base files from framebuffer github were implemented and edited to match our complexity. We used custom sections mostly made without AI for the screen and linked it to our backend logic in order to have a dynamic screen that only refreshes based on changes to the 2D array.

User Guide

- First thing is that both picos need to be connected to the same wifi, preferably a hotspot. After connecting them to the wifi, the server pico will output an IP address to the serial monitor which should be written in the client pico.
- After flashing, the server and client connect to wifi, and then the client connects to the server.
- Each player can press the place button to enter the placement phase. The players can move around the grid with the joystick and can rotate and place their ships using the 2 buttons.
- The system waits for both players to place all their ships, and only after that it enters the attacking phase.
- The server or player 1 gets the first turn and will be able to move around the cursor to attack.
- If a player hits a ship, they get another turn of attack. If they miss, the turn switches to the other player, Player 1 to Player 2 or Server to Client.
- Both players keep playing until all the ships on one grid are hit.
- The game ends, and the winner and loser is displayed.

Hardware Components Used

The following list is for 1 setup. You will need 2 identical setups to build this system:

- Raspberry Pi Pico 2WH
- iCESugar-Pro FPGA
- 2 Buttons
- 2 1kOhm resistors
- 1 Joystick
- 4.3" 40-pin TFT Display - 480x272
- PMOD RGBLCD Expansion Board
- Wires to connect everything

Software Libraries Used

- **pico_stdlib:**
Standard pico library for cmake and other basic pico needs.
- **hardware_spi:**
SPI used to communicate between the pico and the FPGA.
- **hardware_adc:**
To read analog values from the joystick.
- **pico_cyw43_arch_lwip_poll:**
Used to connect the pico to wifi.
- **lvgl::lvgl:**
To create framebuffers to update certain parts of the screen.

Protocols Used

- **SPI:**
Pico handles the game logic and wifi communication. The FPGA handles displaying gameplay based on the game logic. The pico and FPGA communicate using SPI, in order to update the screen to show the current state of the game.

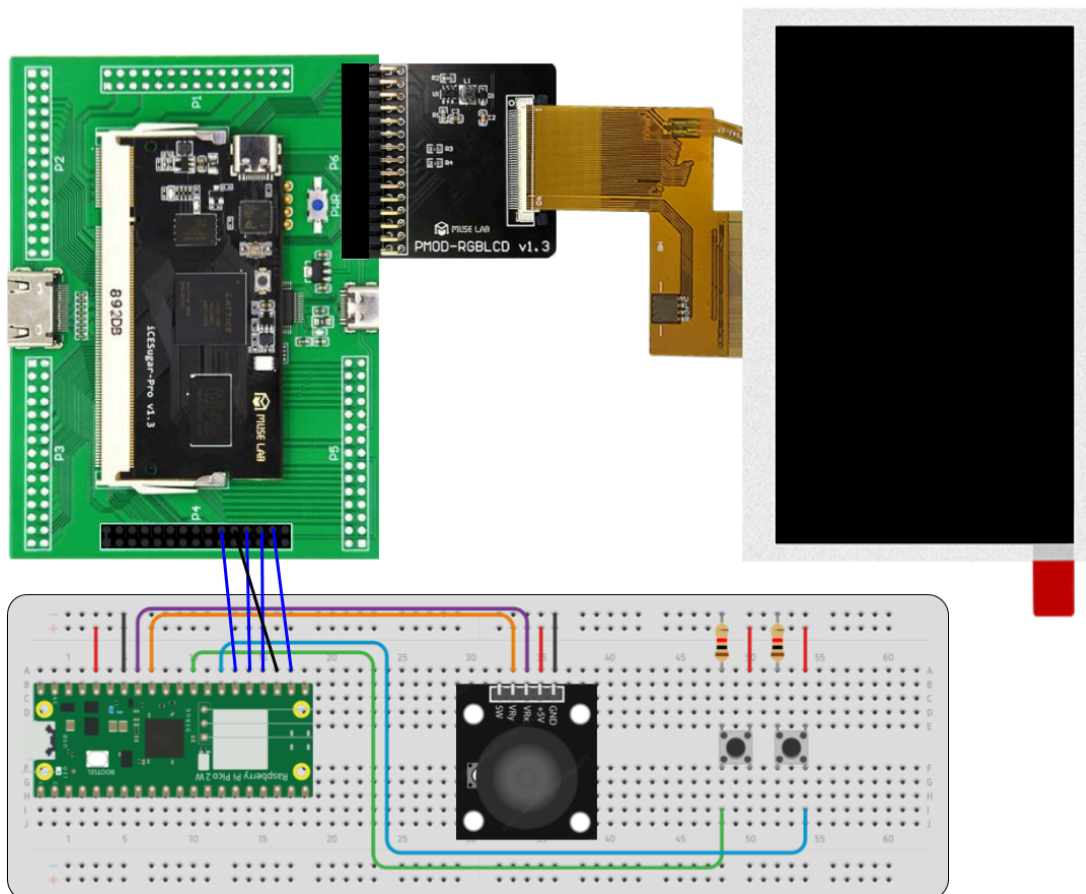
Requirements

List the 4 requirements mentioned in the proposal page and explain what you did for each one.

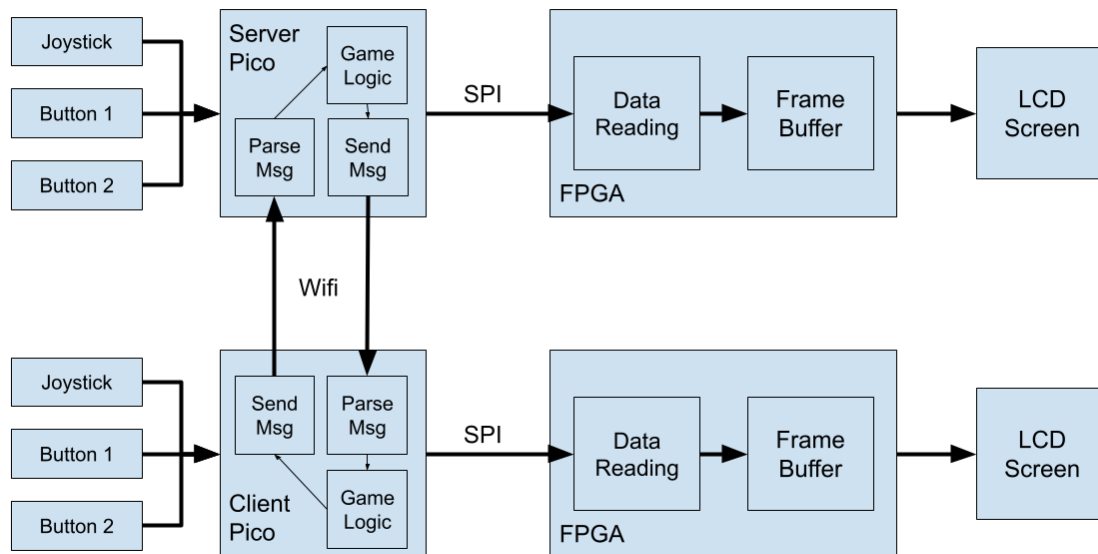
- **Raspberry Pi Pico:**
Utilized for input detection like with buttons and hardware adc, and communicates through FPGA via SPI. Also uses LVGL for partial frame buffering to FPGA so less data is handled by our limited memory.

- **Icesugar-pro FPGA :**
Utilized the FPGA handling SPI data from Raspberry Pi Pico and transferring that data to frame buffer. The frame buffer is then sent to the screen to actively update pixels to the screen.
- **RGB LCD:**
Utilized for gathering the translated data from the FPGA to the PMOD RGBLCD Expansion Board. Outputs the frame buffer to the screen and showcases frames of our game as users can move and select things based on the input provided.
- **Event driven execution :**
Utilized by our Pico as it gathers user input and if Pico even connects to Wifi properly. The game can't even start without proper connection to Wifi and after starting it gathers user data to run and transition to the different stages of the game.

Wiring Diagram



Design Diagram



AI Usage

State what parts of your project you used AI for(if any) and what AI helped you accomplish in those parts

- **Debugging:**

We used AI to try to debug problems if we couldn't find solutions ourselves. There were instances where it was useful and others where it was not.

- **Wifi:**

The raspberry pico has a lot of documentation and resources online but information about how to set up wifi was very difficult to find. Most of the code relating to wifi use was in micropython, which was not useful to us, as we are using C and C++.

- **LVGL:**

AI was utilized a lot here to understand the provided library and library context for our project. Custom screen designs were not made by AI as they are conscious decisions for output, but creating them required AI checks before building. AI was also used in connecting LVGL to a working backend project that could be viewed as a simplified board layout using LEDs.

Acknowledgements

<https://www.raspberrypi.com/documentation/microcontrollers/pico-series.html>

- We used this when researching how to connect to wifi with the pico.

<https://github.com/lvgl/lvgl>

- We used the example layout grid code as reference to make our grid on the screen.

<https://github.com/UCR-CS122A/icesugar-pro-framebuffer>

- Used the code on here to display things on the screen.